```
"""Contextual risk-scoring engine (Framework Domain 4: risk-based authentication)

Takes an access request with five contextual factors and returns one of:
PERMIT, STEP-UP MFA REQUIRED, DENY.

Research prototype only: synthetic data, no live systems.
"""
from dataclasses import dataclass, field
from datetime import datetime

PERMIT = "PERMIT"
STEP_UP = "STEP-UP MFA REQUIRED"
DENY = "DENY"

# Factor weights (points added to the risk score)
ROLE_POINTS = {"standard": 0, "elevated": 15, "privileged": 25}
DEVICE_POINTS = {"compliant": 0, "unmanaged": 15, "non_compliant": 30}
LOCATION_POINTS = {"usual": 0, "new_country": 15, "high_risk": 30}
TIME_POINTS = {"business": 0, "after_hours": 10, "night": 15}

# Score thresholds
STEP_UP_THRESHOLD = 30   # score >= 30 -> step-up MFA
DENY_THRESHOLD = 60      # score >= 60 -> deny

# Hard rules that override the score
BRUTE_FORCE_FAILURES = 5


def failed_attempt_points(failures: int) -> int:
    if failures <= 0:
        return 0
    if failures <= 2:
        return 10
    if failures <= 4:
        return 30
    return 40


def time_band(ts: datetime) -> str:
    """Business = Mon-Fri 08:00-17:59. Night = 22:00-05:59. Otherwise after hours
    if ts.hour >= 22 or ts.hour < 6:
        return "night"
    if ts.weekday() >= 5 or ts.hour < 8 or ts.hour >= 18:
        return "after_hours"
```

```
    return "business"


@dataclass
class AccessRequest:
    user_role: str            # standard | elevated | privileged
    device_posture: str       # compliant | unmanaged | non_compliant
    location: str             # usual | new_country | high_risk
    access_time: datetime
    failed_attempts: int      # recent failed authentications

    def validate(self) -> None:
        checks = [
            (self.user_role, ROLE_POINTS, "user_role"),
            (self.device_posture, DEVICE_POINTS, "device_posture"),
            (self.location, LOCATION_POINTS, "location"),
        ]
        for value, allowed, name in checks:
            if value not in allowed:
                raise ValueError(f"{name} must be one of {sorted(allowed)}, got {
        if not isinstance(self.failed_attempts, int) or self.failed_attempts < 0:
            raise ValueError("failed_attempts must be a non-negative integer")


@dataclass
class Decision:
    decision: str
    score: int
    breakdown: dict = field(default_factory=dict)
    reasons: list = field(default_factory=list)


def evaluate(req: AccessRequest) -> Decision:
    req.validate()
    band = time_band(req.access_time)
    breakdown = {
        "user_role": ROLE_POINTS[req.user_role],
        "device_posture": DEVICE_POINTS[req.device_posture],
        "location": LOCATION_POINTS[req.location],
        "time_of_access": TIME_POINTS[band],
        "auth_behaviour": failed_attempt_points(req.failed_attempts),
    }
    score = min(sum(breakdown.values()), 100)
    reasons = [f"{k}={v}" for k, v in breakdown.items() if v]

    # Hard-deny rules (override score)
    if req.failed_attempts >= BRUTE_FORCE_FAILURES:
```

```
    return Decision(DENY, score, breakdown, reasons + ["rule: suspected brute
if req.user_role == "privileged" and req.device_posture == "non_compliant":
    return Decision(DENY, score, breakdown, reasons + ["rule: privileged acce
if req.user_role == "privileged" and req.location == "high_risk":
    return Decision(DENY, score, breakdown, reasons + ["rule: privileged acce

# Score-based decision
if score >= DENY_THRESHOLD:
    return Decision(DENY, score, breakdown, reasons + ["score at or above den
if score >= STEP_UP_THRESHOLD:
    return Decision(STEP_UP, score, breakdown, reasons + ["score at or above

# Privileged access is never permitted without MFA (least-privilege / PAM con
if req.user_role == "privileged":
    return Decision(STEP_UP, score, breakdown, reasons + ["rule: privileged a

return Decision(PERMIT, score, breakdown, reasons or ["no risk indicators"])
```